

Sistema di Monitoraggio delle Performance dei Judoka: Una Architettura Ibrida basata su un Buffer ibrido Edge-Cloud per l'Analisi Neuromuscolare e Cardiovascolare degli Atleti.

1. Sommario

Contesto e problema Principale

Il monitoraggio sistematico delle prestazioni atletiche richiede la raccolta periodica di KPI biometrici ad alta frequenza in ambienti operativi a bassa digitalizzazione come palestre, tatami e campi di allenamento, dove l'infrastruttura IT è tipicamente assente o ridotta al minimo. Nel contesto specifico di questo progetto, tale esigenza si scontra con tre vincoli strutturali simultanei: la necessità di un'interfaccia di data entry, utilizzabile direttamente sul campo da tecnici in maniera pratica e intuitiva tramite smartphone; la necessità di conformarsi al GDPR sulla gestione di dati biometrici e sanitari, preferendo l'esclusione di soluzioni cloud commerciali standard e un'infrastruttura intermittente per evitare di tenere un PC-server costantemente acceso solo in attesa di un inserimento dati a fine sessione.

Il problema di dominio richiede di consentire l'inserimento rapido di dati strutturati da smartphone a bordo tatami, azzerando i costi infrastrutturali, evitando l'installazione di applicazioni terze e garantendo nativamente la privacy.

La soluzione

Il sistema progettato risponde a questi vincoli con un'architettura ibrida a tre livelli, basata sul disaccoppiamento tra il momento in cui il dato viene digitato e il momento in cui viene effettivamente elaborato e salvato nel database master.

Il primo livello è un Edge Client HTML/JS autonomo e stateless, salvato nella memoria del telefono del tecnico. Non contiene elenchi di atleti e non sa a chi appartengano i dati inseriti.

Un'istanza Supabase funge da buffer temporaneo. Sfruttando le Row Level Security (RLS) di PostgreSQL, l'edge client è vincolato alla sola operazione di INSERT per i payload pseudonimizzati, precludendo a livello di database qualsiasi operazione di lettura o manipolazione dei log.

Il terzo livello è il Master Data Store locale: un database SQLite locale sul PC del responsabile dei dati, configurato in modalità Write-Ahead Logging (WAL) per garantire la massima robustezza e ACIDità. Il PC si accende, interroga Supabase tramite uno script Python, scarica i pacchetti, associa i PIN ai veri nomi degli atleti e pulisce immediatamente il cloud.

Business & Technical Value

L'architettura produce tre risultati misurabili:

- Pseudonimizzazione ad alto isolamento: Nessun dato nominativo viene esposto sul cloud. Il sistema si basa su un rigoroso disaccoppiamento dei dati, dove il dizionario di de-anonimizzazione che collega il PIN all'identità risiede esclusivamente sul Master Data Store locale.
- Costo Infrastrutturale Pari a Zero: Sfruttando Supabase esclusivamente come buffer transitorio (i dati restano online pochi minuti prima di essere scaricati e cancellati), il sistema rientra ampiamente nei tier gratuiti del servizio.
- Attrito di Adozione Inesistente: Il front-end si apre con interfaccia browser dello smartphone. Non ci sono account da creare, aggiornamenti o ulteriori possibili problemi legati a un App terza, compresa la poca intuitività e i primi attriti.

Core Tech Stack

Python · SQLite (WAL) · Supabase (PostgreSQL + Row Level Security) · Streamlit · Tailscale (Mesh VPN) · HTML5/JS (Vanilla + Supabase JS SDK)

2. Requisiti di Business, Vincoli e Contesto Operativo

Vincoli Ambientali

Il sistema verrà utilizzato durante un test in-locò (palestra) per monitorare in tempo reale i risultati dei vari test. L'interfaccia di input deve quindi essere progettata attorno a questo contesto fisico: interfaccia intuitiva, schermo piccolo, inserimento numerico rapido, riscontro immediato sulla trasmissione avvenuta. Parallelamente, il sistema deve supportare un'architettura server-client in cui il server (il PC del responsabile) non è sempre disponibile per questioni energetiche o di interruzioni comunicazione con il Server (che nel caso d'uso effettuato è un PC). Il data entry deve quindi poter avvenire in modo asincrono rispetto alla sincronizzazione sul database master.

GDPR

Le metriche raccolte dal protocollo, come altezze di salto, battiti cardiaci sotto sforzo e tempi di reazione, rientrano nella categoria di dati biometrici e dati relativi alla salute ai sensi del Regolamento UE 2016/679 (GDPR). Trattare questi dati identificando gli atleti su piattaforme cloud commerciali standard (AWS, Google Cloud, Azure) richiederebbe la stipula di accordi DPA (Data Processing Agreement) specifici, e la gestione continuativa della conformità; un overhead legale e organizzativo sproporzionato per un sistema di monitoraggio sportivo a uso interno.

Il progetto aggira il problema eliminando il rischio alla radice tramite un approccio Privacy by Design: sul cloud non transitano mai dati anagrafici o non pseudo anonimizzati. La corrispondenza biunivoca tra il PIN e identità dell'atleta esiste esclusivamente nel database locale, sotto il controllo diretto di chi somministra e monitora il test.

Costi ed Efficienza dell'Infrastruttura

Una soluzione convenzionale basata su server cloud dedicato (EC2, Cloud Run, o equivalenti) con database managed (RDS, Cloud SQL) comporterebbe costi ricorrenti mensili anche per un'istanza minimale, oltre alla necessità di gestione della sicurezza dell'infrastruttura, backup, aggiornamenti e monitoring. Per un sistema a uso interno con un singolo utente concorrente e volumi di dati nell'ordine dei megabyte, questo approccio è ingegneristicamente sovradimensionato.

Il sistema adottato opera su tier gratuiti sia per Supabase (buffer temporaneo) che per Streamlit (dashboard locale), con nessun costo ricorrente o gestione infrastrutturale. L'unica risorsa hardware dedicata è il PC del responsabile, già presente e utilizzato per altre attività, il cui consumo energetico è ottimizzato proprio dall'architettura ibrida: il server locale viene acceso solo quando necessario per la sincronizzazione e l'analisi.

3. Metodologie e Analisi Biomeccanica: Il Framework utilizzato per le stime

Prima di analizzare l'infrastruttura tecnica, è necessario definire il dominio applicativo del sistema. La piattaforma non opera come un semplice raccoglitore di dati, ma implementa un protocollo di valutazione funzionale specifico per il judo progettato per contesti a budget ridotto, eludendo la necessità di strumentazioni costose come le pedane di forza.

Le metriche generate non costituiscono misure dirette di grandezze fisiologiche isolate, bensì *proxy* quantitative derivate: algoritmi che stimano capacità latenti non osservabili direttamente sul campo (potenza meccanica, fatica del sistema nervoso centrale ed efficienza metabolica) a partire da variabili fisiche misurabili sul tatami.

3.0 Overview del protocollo di testing

Una sessione di test si articola in quattro fasi sequenziali, disposte secondo un ordine logico strutturato per minimizzare l'interferenza tra le componenti neuromuscolari e quelle metaboliche:

- Fase 1 - Valutazione neuromuscolare (SJ + CMJ): L'atleta esegue due prove di Squat Jump e due prove di Countermovement Jump (CMJ). Le altezze vengono ricavate dal tempo di volo tramite un'applicazione mobile di video-analisi esterna e inserite immediatamente nell'interfaccia.
- Fase 2 - Velocità di proiezione: Vengono eseguite tre proiezioni tecniche standard cronometrate (in millisecondi), con avvio del tempo rilevato dal momento in cui il primo piede di appoggio tocca il terreno.
- Fase 3 – Protocollo: L'atleta esegue un protocollo Tabata standardizzato (8 × 20s lavoro / 10s recupero) indossando una fascia cardio. Si registra la frequenza cardiaca a inizio, fine, e ai minuti 1, 2 e 3 del recupero.
- Fase 4 - SJTF (Specific Judo Training Factor): Esecuzione di proiezioni continue contro due partner (*uke*) alternati, suddivisa in quattro blocchi temporizzati asimmetrici (15s – 30s – 30s

– 15s). Si registra il numero di proiezioni per blocco e la frequenza cardiaca a inizio, metà, fine e a 1 minuto post-test.

3.1 Valutazione Neuromuscolare e Diagnostica del SNC (Fasi 1 e 2)

Squat Jump (SJ) e Countermovement Jump (CMJ)

Lo Squat Jump è un salto verticale eseguito da una posizione statica con ginocchia flesse a 90°, eliminando il contributo del pre-stiramento muscolare. Valuta quindi la capacità di produrre forza esplosiva concentrica pura, legata alla componente contrattile del muscolo e alla capacità di reclutamento rapido delle unità motorie veloci.

Il Countermovement Jump aggiunge una rapida flessione preliminare prima del salto, attivando il ciclo di stiramento-accorciamento (Stretch-Shortening Cycle, SSC). I tendini e le strutture elastiche del muscolo immagazzinano energia elastica durante la fase eccentrica, restituendola amplificata nella successiva spinta concentrica. In condizioni fisiologiche normali, la prestazione nel CMJ supera sistematicamente quella nel SJ.

Indice di Efficienza SSC

L'efficienza del ciclo stiramento-accorciamento viene calcolata tramite il rapporto:

$$SSC_{Efficiency} = \frac{Max_{CMJ}}{Max_{SJ}}$$

Un valore prossimo o inferiore a 1.0 evidenzia l'incapacità di sfruttare la componente elastica, indice di fatica neuromuscolare acuta o deficit pliometrici. La letteratura biomeccanica identifica come ottimale un intervallo compreso tra 1.05 e 1.15. Valori superiori a 1.15-1.20 indicano un'eccellente efficienza elastico-riflessiva, ma possono anche segnalare una carenza strutturale nella forza concentrica di base.

Z-Score CMJ e Prevenzione Infortuni

Il CMJ si dimostra altamente sensibile ai cali di efficienza del sistema nervoso centrale prima che la fatica emerga a livello sintomatico o infici la coordinazione tecnica. Un deficit nel reclutamento delle fibre veloci si traduce in un calo immediato dell'altezza del salto, rendendo la metrica un indicatore predittivo del rischio di infortunio da sovraccarico.

Il sistema implementa un algoritmo di Z-Score storico su finestra mobile. Un calo della prestazione con Z-Score compreso tra -1,0 e -1,5 indica affaticamento acuto. Un decremento pari o inferiore a

$Z < -1,5$ definisce la soglia critica di allerta per staff tecnico, indicando probabile sovrallenamento e necessità di modulare il carico per prevenire lesioni. Effettuare un test dopo un breve riscaldamento muscolare permetterà di individuare e gestire meglio tali problematiche.

Stima della Potenza di Picco

Per sopperire all'assenza di pedane dinamometriche, il sistema integra la formula cinetica di Sayers (1999), validata per la stima della potenza di picco nei salti verticali:

$$\text{Picco Potenza (W)} = 60.7 \times \text{Altezza salto (cm)} + 45.3 \times \text{Massa corporea (kg)} - 2055$$

Il software calcola la potenza assoluta (W) e la potenza relativa (W/kg), parametro fondamentale per il confronto oggettivo tra atleti appartenenti a categorie di peso differenti e per il monitoraggio longitudinale della forza dinamica, appoggiandosi a un tracciamento aggiornato del peso corporeo in anagrafica.

3.2 Prestazione Cardiovascolare e Meccanica (Fasi 3 e 4)

Capacità Funzionale ad Alta Intensità (Tabata)

Il protocollo Tabata (1996) viene impiegato come stimolo standardizzato per valutare la risposta autonoma del sistema cardiovascolare. La metrica d'interesse è la curva di ripristino della frequenza cardiaca (*Heart Rate Recovery*, HRR) nelle fasi successive allo sforzo massimale.

Il decremento della HRR al primo minuto post-test rappresenta un indicatore della riattivazione del sistema nervoso parasimpatico. Calcolando i delta percentuali e assoluti a 1, 2 e 3 minuti, la dashboard traccia il profilo di recupero dell'atleta, fornendo indicazioni sulla capacità di smaltimento del carico metabolico tra i match di una competizione.

Specific Judo Fitness Test (SJTF)

Il SJTF è un protocollo di valutazione della resistenza speciale nel judo che ho sviluppato specificamente per replicare le sollecitazioni e i regimi energetici intermittenti tipici del combattimento. L'atleta esegue proiezioni standard continue contro due uke alternati a distanza di 6 metri, strutturate in quattro blocchi temporizzati asimmetrici (15s – 30s – 30s – 15s) con interruzioni di recupero di 10s – 15s – 15s che replicano l'alternanza di fasi ad alta e media intensità tipica di un incontro di judo. Vengono raccolte il numero di proiezioni per blocco e la frequenza cardiaca all'inizio, a metà, alla fine e a 1 minuto post-test.

Il sistema calcola due metriche derivate:

- $$SJTF = \frac{\Sigma HR}{\Sigma Proiezioni'}$$

ovvero il rapporto tra il carico interno (stress cardiovascolare) e l'output prodotto. Esprime il costo cardiocircolatorio per singola tecnica eseguita. Un aumento dell'indice riflette una perdita di efficienza neuro-meccanica o un calo della resistenza specifica.

- $$Drop - Off (\%) = \frac{Proiezioni B1 - Proiezioni B4}{Proiezioni B1} \times 100,$$

misura il decadimento delle prestazioni tra il primo e l'ultimo blocco di 15 secondi. Quantifica la gestione e la tolleranza della fatica dell'atleta, così come la sua capacità di mantenere costante l'output sotto uno stress prolungato. Un Drop-Off elevato (si effettuano circa 5-9 tecniche in 15 secondi) indica difficoltà nel sostenere l'intensità nella fase finale del test, con implicazioni dirette sulla strategia di gara.

4. Architettura e Analisi Comparativa

4.1 Valutazione delle alternative

Prima di convergere sull'architettura adottata, sono state valutate quattro alternative. L'analisi dei loro limiti è parte integrante della giustificazione delle scelte progettuali finali.

On-Device Local Server (Termux/Pydroid)

L'idea di far girare un server Python (Flask o FastAPI) direttamente sullo smartphone, accumulando i dati in locale per poi scaricarli sul PC, si scontra con la realtà dei sistemi operativi mobili. I runtime Python o gli emulatori Linux su Android (Termux, Pydroid) operano in ambienti non standardizzati e penalizzati dalle recenti policy di risparmio energetico dei vendor, che tendono a uccidere i processi in background e a isolare il traffico di rete. Inoltre, questa soluzione avrebbe richiesto configurazione e manutenzione tecnica su ogni singolo telefono, violando il requisito di adozione immediata e senza attrito.

Native Cross-Platform App + Cloud Backend (AWS/PostgreSQL)

La soluzione architetaturalmente più "standard", ovvero un'applicazione React Native o Flutter con backend che si interfaccia con un database transazionale sempre attivo su cloud (e.g. AWS) è palesemente sovradimensionato per questo contesto. I costi e i tempi di sviluppo e manutenzione di un'app nativa sono significativi. Il backend introduce costi ricorrenti e, soprattutto, richiede l'archiviazione di dati biometrici identificabili su server di terze parti, ricadendo direttamente nei

vincoli GDPR descritti nella sezione 2.2. Questa opzione è stata scartata per eccesso di complessità e non conformità normativa strutturale. Un'app nativa per smartphone che si comporta similmente al file HTML (scrittura cifrata su Supabase) o che registra ogni dato offline sullo smartphone permettendo di inserire e visualizzare risultati da mobile è sicuramente la migliore soluzione possibile per questi requisiti, ma in questo progetto ho voluto esplorare altre possibilità.

Micro-Hardware Always-On Dedicato (Raspberry Pi)

Un Raspberry Pi come server locale always-on risolve il problema della disponibilità del server senza richiedere il PC principale sempre acceso, consumando un'energia drasticamente inferiore. Tuttavia introduce nuove dipendenze come costo di approvvigionamento hardware, la gestione dell'aggiornamento del sistema operativo. Non risolve il problema dell'inserimento dati in mobilità senza rete locale.

Thin Client con Cache + FastAPI

Una variante intermedia prevede un'interfaccia mobile che memorizza temporaneamente i test nella cache del browser e scarica i pacchetti verso un backend FastAPI non appena rileva una connessione attiva. Sebbene valida, questa architettura introduce una complessità non banale nella gestione dello stato offline/online e nella risoluzione dei conflitti in caso di scritture concorrenti. Rispetto al modello ibrido poi adottato, non offre vantaggi reali a fronte di una logica di sincronizzazione molto più onerosa da scrivere e testare.

4.2 Modalità Operativa Sincrona - Edge-to-Premise via VPN Mesh

La prima architettura implementata opera in modalità sincrona: il dispositivo mobile si connette direttamente al PC locale tramite una rete VPN mesh e accede alla dashboard Streamlit come se fosse sulla stessa rete locale.

Cos'è Tailscale e la rete mesh

Tailscale è un servizio VPN basato sul protocollo WireGuard che stabilisce una rete privata virtuale in topologia mesh peer-to-peer. A differenza delle VPN tradizionali (hub-and-spoke), che costringono il traffico a transitare da un server o gateway centrale, la rete mesh permette a ogni nodo di connettersi direttamente agli altri. La cifratura è end-to-end a livello di trasporto

(ChaCha20-Poly1305) e i server di coordinamento di Tailscale gestiscono solo lo scambio iniziale delle chiavi e degli indirizzi IP, senza mai poter decifrare i dati.

Nel concreto, Tailscale assegna al PC master un IP privato stabile, rendendolo raggiungibile dallo smartphone anche quando i dispositivi si trovano su reti fisiche e geografie totalmente diverse (es. PC collegato alla rete di casa e smartphone agganciato al 5G dalla palestra). Questa configurazione permette di saltare completamente procedure complesse come il port forwarding sul router o la sottoscrizione di servizi DNS dinamici (DDNS), senza esporre alcuna porta sul perimetro internet pubblico.

Server-Side Rendering via Streamlit

Con Tailscale attivo, il browser dello smartphone raggiunge direttamente l'applicazione Streamlit in esecuzione sul PC. Streamlit opera come server. Lo smartphone non esegue alcuna logica applicativa, ma è un thin client puro che visualizza e trasmette input. Questo approccio elimina qualsiasi problema di compatibilità del browser mobile con librerie Python, garantendo che la scrittura sul database avvenga esclusivamente sull'hardware controllato.

Trade-off dell'Architettura A

Il vantaggio evidente è la sovranità assoluta sul dato: le metriche vengono iniettate direttamente nel database master in tempo reale, senza tappe intermedie. Lo svantaggio è il vincolo di disponibilità sincrona: il PC deve essere acceso, connesso a internet e con l'applicazione Streamlit in esecuzione nel preciso istante in cui l'atleta salta. In caso di blackout, disconnessioni della linea di casa o assenza fisica del responsabile del PC, il sistema diventa inutilizzabile. Tale vincolo hardware è aggirabile configurando il BIOS della macchina master per l'accensione automatica al ripristino dell'alimentazione attivata da una presa smart, combinata con l'esecuzione dello script Python all'avvio del sistema operativo. Lo spegnimento può invece essere pilotato da remoto tramite l'interfaccia Streamlit.

4.3 Modalità Operativa Asincrona - Hybrid Cloud-Buffer Pattern

Per ovviare il vincolo temporale e hardware, la modalità asincrona introduce un disaccoppiamento esplicito: la raccolta del dato sul campo è slegata dalla sua validazione e persistenza nel DB sul PC. Non sono due architetture distinte, ma vanno coordinate per inserire i dati in modalità asincrona (pseudoanonimizzati) e visualizzarli/inserire atleti solo con contatto VPN mesh tramite Tailscale quando necessario.

Zero-Knowledge Edge Client

Il front-end di inserimento è ridotto a un file statico HTML/JS. È un client leggero, privo di logica server-side, che può essere salvato localmente nella memoria del telefono ed eseguito da qualsiasi browser. Il codice non contiene elenchi di atleti o tabelle anagrafiche: l'identificazione avviene unicamente digitando un PIN a 4 cifre.

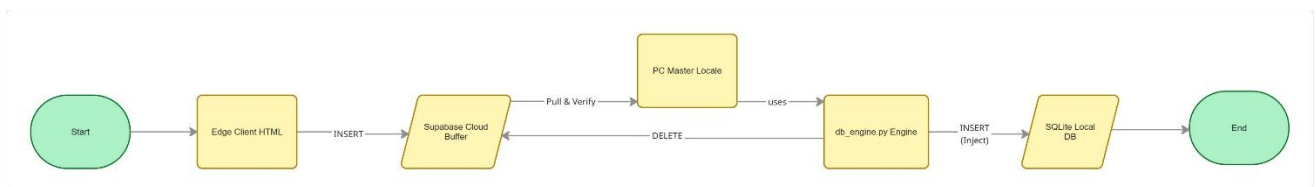
Pipeline Effimera sul Cloud (Supabase)

In questa configurazione, Supabase, non funge da storage, ma come coda di transito cieca. Le tabelle remote (buffer_salti, buffer_completo) accettano unicamente payload pseudonimizzati (PIN + metriche).

La sicurezza logica è governata dalle Row Level Security (RLS) del database: il client HTML possiede una chiave pubblica anonima con privilegi limitati al solo comando INSERT. Non è possibile effettuare letture (SELECT), modifiche o cancellazioni dal telefono. Questo garantisce che anche in caso di compromissione del client HTML, un attaccante potrebbe solo inserire dati nel buffer, mai leggere o cancellare quelli esistenti.

Protocollo di Ingestione e Sincronizzazione dei Dati

Quando il PC è disponibile, il responsabile avvia manualmente (o programmaticamente) il ciclo di sincronizzazione:



Il backend interroga Supabase tramite REST API e scarica tutti i record presenti nel buffer. Per ogni record, normalizza il timestamp, verifica l'integrità referenziale (PIN già presente) e tenta l'inserimento nel database SQLite locale. Solo dopo la conferma dell'inserimento locale avvenuto con successo, il record viene marcato per la cancellazione dal cloud. La cancellazione viene eseguita in batch (unicamente per scalabilità, non necessaria nelle attuali condizioni). Al termine del ciclo, il buffer cloud è vuoto.

Gestione dell'errore

Per garantire la tolleranza al fallimento senza perdere dati, il sistema implementa una strategia di isolamento (Dead Letter Queue): i record con PIN non riconosciuto vengono deviati in una tabella temporanea, che conserva il JSON grezzo per revisione manuale. Il record viene comunque cancellato dal cloud, garantendo continuità nel ciclo di sincronizzazione, assicurando che nessun dato valido venga perduto per colpa di un record mal formulato.

Inoltre, implementare controlli sui dati inseriti per evitare errori di digitazione è possibile (e.g. limitare HR a un determinato intervallo).

5. Profilo di sicurezza e conformità per progettazione

La sicurezza del sistema non è implementata come uno strato applicativo aggiuntivo, ma emerge come proprietà strutturale del design stesso. I tre pilastri descritti di seguito - pseudonimizzazione, controllo degli accessi e minimizzazione della persistenza - sono conseguenze dirette del disaccoppiamento architetturale.

5.1 Pseudoanonimizzazione e Disaccoppiamento dell'identità

Il meccanismo centrale di conformità GDPR del sistema è la separazione logica e fisica tra il dato biometrico e l'identità dell'atleta. L'unico punto di contatto tra le metriche fisiche e l'atleta è un PIN numerico a 4 cifre: un identificatore opaco che, privato del dizionario di de-anonimizzazione, non possiede alcun potere identificativo. Il dizionario di accoppiamento risiede esclusivamente all'interno del database SQLite installato sulla macchina locale del responsabile del trattamento.

Questo approccio traduce in codice il principio di Data Protection by Design and by Default (Art. 25 GDPR), assicurando che le due metà dell'informazione non coesistano mai nello stesso spazio logico.

5.2 Ottimizzazione del Controllo nell'Accesso al Cloud

Il controllo degli accessi al layer cloud è implementato tramite le policy Row Level Security (RLS) di PostgreSQL, abilitate nativamente su Supabase. La configurazione adottata realizza un principio di privilegio minimo applicato per chiave:

La chiave pubblica anonima (distribuita nel file HTML del client mobile), come detto precedentemente è autorizzata esclusivamente all'operazione di INSERT sulle tabelle buffer. Non può leggere i record esistenti, non può modificarli, non può cancellarli.

La chiave privata, che ha privilegi completi di lettura, scrittura e cancellazione, è conservata esclusivamente nel file .env sull'ambiente on-premise, non distribuita e non appare in nessun file accessibile lato client. L'elevazione dei privilegi avviene quindi solo nell'ambiente controllato, durante il ciclo di sincronizzazione.

5.3 Data Minimization e Protocollo di Cancellazione

Il terzo livello di protezione consiste nel contrarre la finestra temporale di persistenza delle metriche sul cloud. Le tabelle di Supabase non fungono da storage storico, ma operano come una coda a consumo rapido che viene ripulita ad ogni ciclo di sincronizzazione. Non vengono utilizzati soft delete o flag di stato: il record viene rimosso definitivamente dalla tabella cloud.

In condizioni operative normali, con sincronizzazione post-sessione, i dati pseudonimizzati risiedono su Supabase per poche ore. Anche in caso di mancata sincronizzazione prolungata, i dati esposti sono comunque privi di qualsiasi attributo identificativo diretto.

5.4 Gestione delle vulnerabilità di Data Ingestion

L'esposizione della chiave anonima nel client HTML espone il buffer cloud a potenziali attacchi di tipo spam di Insert. Il rischio è mitigato operativamente in due modi: sul cloud tramite policy di Rate Limiting gestite nativamente a livello di API Supabase, e on-premise tramite il codice Python, che confina i payload con PIN inesistenti in una Dead Letter Queue isolata, prevenendo l'inquinamento del dataset primario.

6. Decisioni di Progettazione Tecnica

6.1 Strategia di immutabilità del dato

Una scelta di design non convenzionale del sistema è l'assenza intenzionale di interfacce di modifica o cancellazione dei record sul front-end mobile. Questa impostazione risponde a precise ragioni sia operative che architetturali:

- Sul piano operativo: Le metriche raccolte (altezze di salto, frequenze cardiache, volumi di proiezioni) sono osservazioni istantanee legate indissolubilmente al momento del test. In caso di inserimento errato, la misurazione originale è comunque persa e correggibile (sempre e comunque tramite arbitrarietà) solo in caso di errori lampanti (e.g. 1800 anziché 180bpm), ma non in caso di errori più celati (e.g. 180 anziché 170bpm).
- Sul piano architetturale, introdurre operazioni di Update o Delete sul client mobile costringerebbe la pipeline asincrona Pull-Inject-Delete a implementare logiche complesse di riconciliazione degli stati conflittuali tra client e master DB. L'immutabilità azzerava questa classe di problemi: ogni record è trattato come un evento autonomo, semplificando la logica transazionale.

Questa è una scelta deliberata di semplicità: il numero di errori significativi atteso è basso, sia per la mole di atleti e test effettuati, sia per l'estrema linearità del test e dell'interfaccia (come a sostituire il classico "foglio di carta". Pertanto la loro correzione è rapida e delegata ad un operatore con competenze tecniche di base con accesso al Database tramite strumenti standard (DB Browser for SQLite), eliminando il costo per l'implementazione di un'interfaccia di modifica sicura e coerente, che supererebbe di gran lunga il beneficio nel contesto d'uso specifico.

6.2 Master Storage Engine

La scelta di adottare SQLite come database master è stata preferita a soluzioni client-server tradizionali (come PostgreSQL o MySQL) sulla base di una valutazione analitica del carico di lavoro e dei volumi d'archivio.

Il volume di dati stimato per una rosa di 20-30 atleti con frequenza di test bisettimanale/mensile si attesta nell'ordine di pochi megabyte anche nell'arco di diversi anni. SQLite elabora dataset di queste dimensioni con performance in lettura e scrittura superiori rispetto a un RDBMS client-server.

L'architettura presenta un singolo processo di scrittura concorrente (il motore di sincronizzazione Python) e un numero limitato di lettori (l'istanza Streamlit). Per risolvere l'unico scenario di concorrenza potenziale, la consultazione della dashboard durante l'iniezione dei dati dal cloud, il database è configurato in modalità Write-Ahead Logging (WAL).

In modalità WAL le operazioni di scrittura vengono registrate in un file di log separato prima di essere conciliate nel database principale, permettendo ai lettori di accedere all'ultimo stato

consistente del file senza subire i blocchi imposti dalle transazioni di scrittura in corso, preservando le proprietà ACID del sistema con un impatto sistemistico nullo.

6.3 Front-End senza attrito

L'interfaccia di inserimento dati è sviluppata in puro codice HTML5 e JavaScript nativo, senza l'ausilio di framework o fasi di compilazione. L'unica dipendenza è il modulo client di Supabase. Questo comporta alcuni benefici:

- Il file HTML viene aperto direttamente nel browser mobile. Gli aggiornamenti all'interfaccia, sporadici, vengono distribuiti semplicemente sostituendo il file HTML sul dispositivo (condividendolo via AirDrop, WhatsApp, e-mail...), permettendo di evitare di dover pubblicare su App Store o Play Store. Questa trafila non è necessaria per un'applicazione ad uso interno con uno o pochissimi operatori.
- Portabilità multiplatforma nativa. Lo stesso file funziona identicamente su qualsiasi sistema operativo e browser moderno, senza necessità di mantenere code base separate per iOS e Android, il costo principale delle app native Cross-Platform.

7. Dashboard e Motore di Analisi Dati

Il motore analitico è implementato in Python e reso accessibile tramite una dashboard Streamlit che opera esclusivamente sull'ambiente on-premise. L'architettura dei moduli è disaccoppiata per praticità di test e di uso.

7.1 Benchmark di Gruppo

La vista di benchmark di gruppo genera una matrice comparativa che aggrega, per ogni atleta, l'ultima sessione disponibile per ciascuna tipologia di test. La join tra i due registri (salti e test completo) garantisce che la tabella mostri sempre l'ultima rilevazione disponibile, indipendentemente dalla frequenza con cui i diversi atleti hanno eseguito i test.

Le metriche calcolate per ogni atleta nella vista di gruppo includono: indice di efficienza SSC, picco di potenza assoluta e relativa per SJ e CMJ, velocità media di proiezione, il supporto e la variazione della HR durante il Tabata e durante il recupero, il SJTF, il numero di proiezioni per blocco, e Drop-Off percentuale.

Questa vista è lo strumento principale per la gestione del gruppo: permette allo staff tecnico di identificare a colpo d'occhio gli atleti con profili di fitness divergenti, pianificare carichi di

allenamento differenziati, e monitorare l'evoluzione collettiva del gruppo nel tempo. La dashboard permette l'esportazione della matrice in formato CSV per analisi esterne o archiviazione.

7.2 Profilo Individuale dell'atleta

La vista individuale analizza le serie storiche del singolo atleta, calcolando i delta assoluti e percentuali rispetto alla sessione d'esame immediatamente precedente, permettendo di identificare la tendenza sia nel breve che nel lungo periodo (tramite apposite dashboard visive). Futuri sviluppi includeranno la possibilità di esportare una dashboard personale alla fine di ogni test completo, da inviare all'utente specifico se lo richiede.

7.3 Prontezza del Sistema Nervoso Centrale e Modello di Rischio Infortunio

Questa è la componente analiticamente con il maggiore impatto diretto sulla prevenzione degli infortuni. Il sistema nervoso centrale, come detto precedentemente, può essere testato osservando la variazione dell'altezza del salto CMJ (attivazione ciclo accorciamento-allungamento) rispetto ai salti svolti nei giorni precedenti [Gathercole et al. (2015)]. Il sistema nervoso centrale è cruciale negli sport di combattimento. Allenarsi ad una certa intensità con un SNC affaticato (definibile come riduzione della capacità di attivare volontariamente il muscolo scheletrico) apporta diversi svantaggi:

- Altera il pattern motorio: il SNC compensa l'incapacità di reclutare le fibre primarie attivando unità motorie secondarie o alterando la sequenza di attivazione muscolare, sporcando la biomeccanica del gesto e fissando engrammi (traccia fisica o biochimica lasciata da un ricordo o da un'esperienza all'interno del cervello) motori errati durante l'apprendimento.
- Fa crollare il tempo di reazione, riducendo capacità anticipatorie e di elaborazione di stimoli visivi e tattili.
- Aumenta il rischio di infortuni poiché la stabilità articolare dinamica è dovuta principalmente da una rapida co-contrazione di muscoli agonisti e antagonisti. Un ritardo nei segnali efferenti dal SNC ai muscoli stabilizzatori lascia l'articolazione vulnerabile.

Architettura dell'algoritmo

Il modulo calcola lo Z-Score individuale applicando una finestra mobile (Rolling Window) impostata su un'ampiezza di 10 sessioni storiche, richiedendo un set minimo di 3 rilevazioni per l'attivazione.

L'algoritmo isola il valore massimo tra i tentativi di CMJ eseguiti nella medesima sessione, assumendo il picco prestativo come l'espressione più fedele della capacità neuromuscolare istantanea.

La serie viene traslata di un'osservazione, garantendo che la media e la deviazione standard utilizzate come baseline per valutare il test odierno siano calcolate esclusivamente sulle sessioni precedenti, escludendo il valore corrente evitando bias sistematici.

Interpretazione

Un valore Z positivo indica una performance superiore alla baseline individuale recente, segnale di picco di forma. Un valore Z negativo indica una performance inferiore alla baseline. La soglia operativa adottata è $Z < -1.5$ deviazioni standard (Probabilità di verificarsi minore del 7%), configurando una condizione di elevata vulnerabilità strutturale a lesioni muscolari o articolari se sottoposti a sforzi elevati.

È fondamentale qualificare correttamente l'output di questo modello: lo Z-Score non è una diagnosi, ma un segnale di allerta per lo staff tecnico. Un valore $Z < -1.5$ non implica necessariamente sovra allenamento o rischio infortunio imminente, ma può riflettere variabilità naturale, condizioni ambientali (temperatura, ora del giorno), o fattori extra-allenamento (qualità del sonno, stress). Il valore del sistema sta nella sistematizzazione del monitoraggio fornendo uno storico quantitativo.

Questo rigore evidenzia perché il termine "stima" sia la chiave di lettura dell'intero framework analitico. Le grandezze biomeccaniche e metaboliche calcolate non sono misure dirette da laboratorio, ma proxy predittive. La validità scientifica del software non risiede nella precisione assoluta del singolo dato estemporaneo, ma nella coerenza, nella standardizzazione e nella ripetibilità dei modelli matematici nel tempo.

8. Conclusioni e Sviluppi Futuri

8.1 Sintesi delle Scelte Progettuali

La piattaforma descrittiva non rappresenta la soluzione più lineare al problema del tracciamento atletico, bensì la configurazione minimale capace di soddisfare simultaneamente i requisiti che mi sono posto in relazione ai vincoli operativi: mobilità sul campo, intermittenza dell'hardware master, conformità nativa al GDPR, assenza di costi di licenza e adozione immediata.

Ogni scelta tecnica come SQLite invece di PostgreSQL, HTML vanilla invece di un'app nativa, Supabase come buffer temporaneo invece di storage permanente, sono il risultato di un trade-off esplicito tra implementazione, requisiti funzionali reali e ciclo di vita del software.

Il sistema risultante è minimale per scelta: la sua semplicità è una proprietà di design, mirata a massimizzare l'affidabilità riducendo i punti di potenziale guasto del sistema.

8.2 Sviluppi Futuri

Scalabilità orizzontale verso discipline affini

Il protocollo di test è attualmente specifico per il judo (e ancora limitato e ampliabile, ad esempio, tramite dinamometro per hand grip) nella scelta delle metriche (proiezioni, SJTF), ma l'architettura è completamente agnostica rispetto al dominio applicativo. L'estensione orizzontale del sistema a discipline caratterizzate da profili metabolici e neuromuscolari sovrapponibili, come il Sambo, il Brazilian Jiu-Jitsu o la Lotta Libera richiederebbe esclusivamente la riscrittura dei form di inserimento e l'aggiunta delle relative tabelle nel database locale. L'architettura core di acquisizione, sicurezza logica e sincronizzazione asincrona rimane invariata.

Applicazione Nativa Mobile

Un'ulteriore evoluzione del sistema prevede la transizione verso un'applicazione mobile nativa o PWA in grado di archiviare temporaneamente i test in locale (tramite una cache o un database on-device come IndexedDB), per poi avviare una sincronizzazione asincrona verso il PC-server master non appena viene attivato il tunnel VPN crittografato di Tailscale. Questa architettura offrirebbe il vantaggio di una totale operatività offline sul tatami, ma introdurrebbe altre limitazioni e problematiche: in primo luogo la necessità di sviluppare tenendo conto dei sistemi operativi in uso e ad eventuali aggiornamenti di sistema (da tenere sotto controllo nel futuro) o esponendo il sistema a rischi di perdita di dati localizzati a causa di wipe della memoria su cui si ha poco controllo (nel caso di cache) o soggetti a furto/smarrimento del dispositivo mobile.